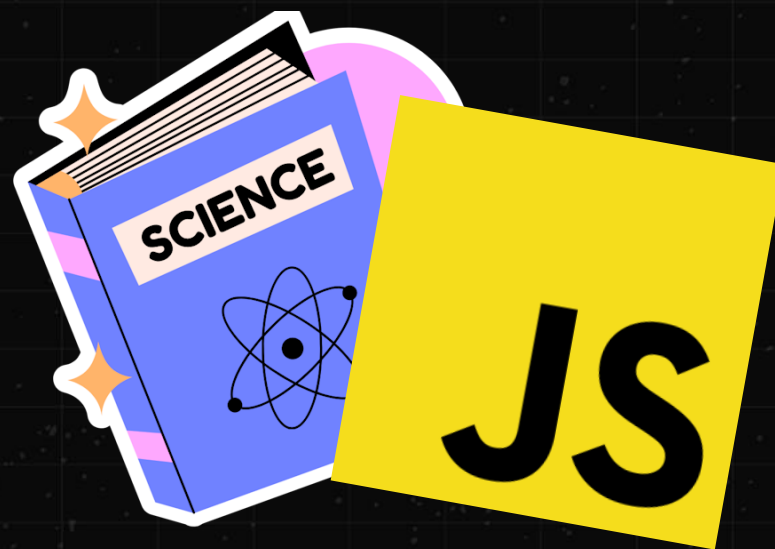


ЛЕКЦИЯ #17

CS BO FRONTEND



ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- * Познакомимся с новой **абстрактной структурой данных** — очередью с приоритетом
- * Детально разберём её **реализацию на основе сбалансированного дерева** — бинарной кучи
- * Поговорим об областях применения таких очередей

ВСПОМНИМ ПРО ОЧЕРЕДЬ

- * Это абстрактная структура данных, которая представляет собой **упорядоченный список значений**
- * Структура поддерживает операции **добавления в конец списка, а также чтение и удаление из начала**
- * Наивно реализовать очередь можно как с помощью связанного списка, так и на основе массива
- * Эффективные реализации улучшают эти подходы, например, с помощью кольцевого буфера на массиве

ОЧЕРЕДЬ С ПРИОРИТЕТОМ

- * Если в обычной очереди **порядок определяется только операциями** вставки и удаления
- * То в очереди с приоритетом **порядок может быть изменён** в зависимости от веса (приоритета) самого элемента
- * Это можно сравнить с очередью на аттракционы, где у части людей есть быстрый или VIP-проход

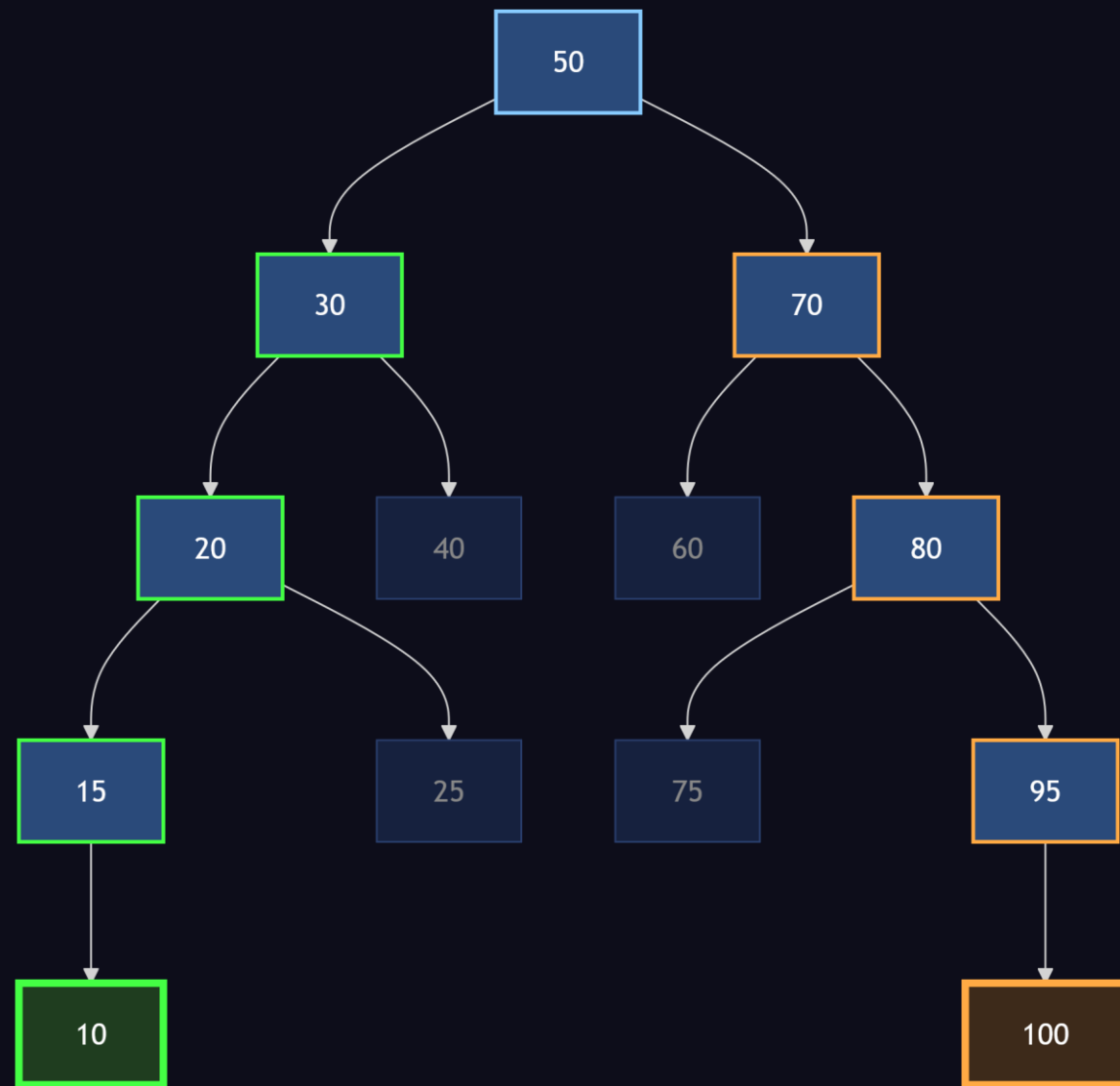
КАК ЭТО **МОЖНО**
РЕАЛИЗОВАТЬ?



НАИВНАЯ РЕАЛИЗАЦИЯ

- * **Отсортированный массив:** первый элемент всегда будет самым маленьким или самым большим — чтение за константу
- * Удаление и вставка **потребуется двигать элементы массива** — $O(n)$, что крайне неэффективно
- * Можно исправить ситуацию, используя **сбалансированные деревья поиска**
- * Любое сбалансированное дерево **может найти минимум и максимум** за логарифмическое время

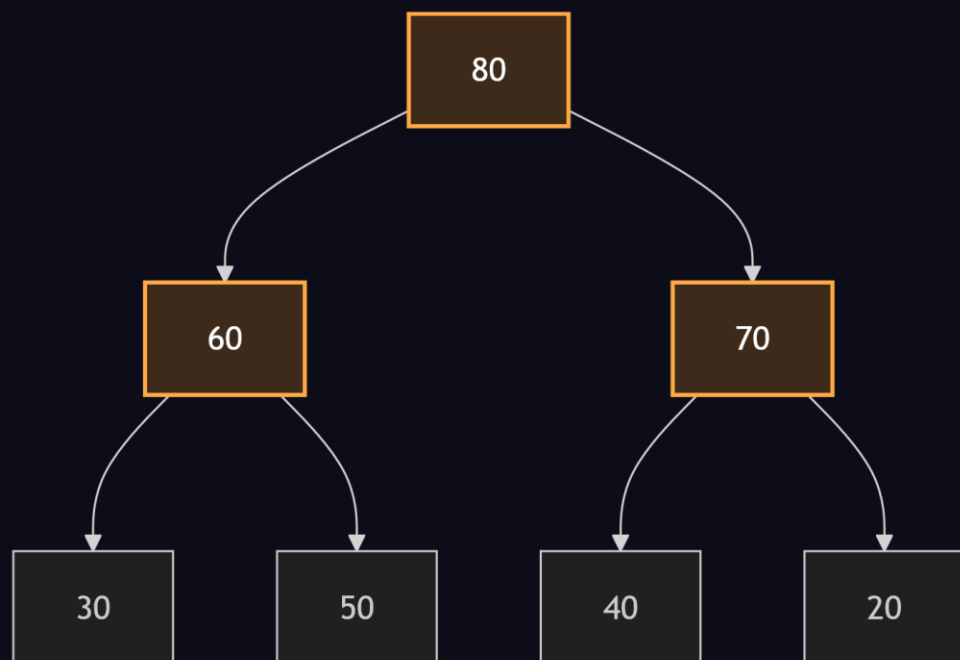
Бинарное дерево поиска



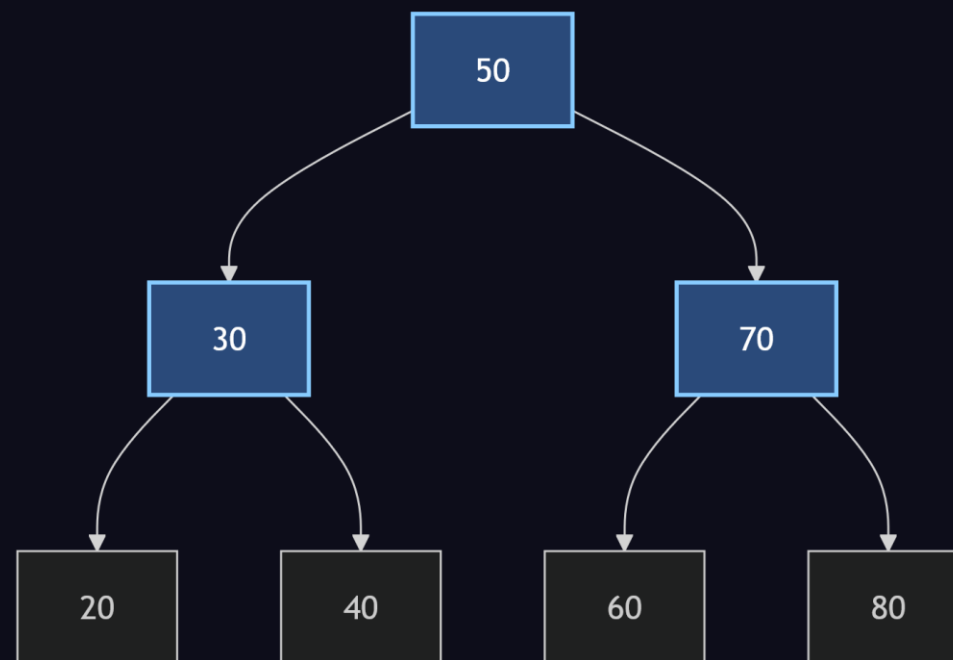
НО МОЖНО ИЗМЕНИТЬ УСЛОВИЕ СОРТИРОВКИ

- * В бинарном дереве поиска **левый узел меньше родителя, а правый больше**
- * Вместо этого можно сделать так, чтобы родитель был **всегда больше или всегда меньше своих потомков**
- * Такое изменение потребует пересмотра операций вставки и удаления
- * При этом мы **потеряем возможность быстро искать** произвольные элементы — это уже не дерево поиска
- * Зато **в корне всегда будет самый большой** (или самый маленький) элемент
- * Операция чтения корня является константной

Мах-упорядоченное



Бинарное дерево поиска



ЗАЧЕМ ЭТО НУЖНО

- * Позволяет максимально **быстро получить элемент с наивысшим приоритетом**
- * Используется в алгоритмах, где **порядок обработки зависит от важности**, а не от времени поступления
- * Поиск кратчайшего пути (алгоритм Дейкстры)
- * Сжатие данных (алгоритм Хаффмана) – построение оптимального префиксного кода
- * Планировщики задач – срочные задачи выполняются раньше
- * Событийные системы (обработка событий с разным приоритетом)
- * И многое другое!

ДЕРЕВО ДОЛЖНО **БЫТЬ**
СБАЛАНСИРОВАННЫМ?



БЕЗУСЛОВНО

- ✱ Ведь кроме операции чтения корня у нас есть **вставка и удаление**
- ✱ Для того чтобы они работали за логарифмическое время, **дерево должно быть сбалансированным**
- ✱ Так как у нас бинарное дерево, мы можем «вписать» его в массив, используя **формулы для вычисления левого и правого потомка**

ВСТРЕЧАЙТЕ, БИНАРНАЯ КУЧА

- * Бинарное частично упорядоченное дерево, реализующее очередь с приоритетом
- * Операции вставки и удаления гарантируют сохранение баланса
- * На основе этой структуры создан один из лучших алгоритмов сортировки – пирамидальная сортировка (сортировка кучей)

ПЕРЕЧИСЛИМ ИНВАРИАНТЫ

- * Корень всегда самый большой (max-heap) или самый маленький (min-heap) элемент
- * **Взятие корня** — **константная** операция $O(1)$
- * Вставка и удаление выполняются **за логарифмическое время** $O(\log n)$
- * Можно эффективно реализовать на основе массива

НЕ ЕДИНСТВЕННАЯ РЕАЛИЗАЦИЯ

- * Очередь с приоритетом **может быть реализована по-разному**
- * Кроме бинарной кучи, существуют такие структуры, как Биномиальная куча или Фибоначчиева куча
- * У каждой структуры есть свои плюсы и минусы
- * Но бинарная куча **в среднем является лучшим кандидатом для большинства задач**

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

НАША ЗАДАЧА НА СЕГОДНЯ

- * Обычно мы ограничиваемся пониманием концепций работы тех или иных структур
- * Но сегодня мы **подробно разберём и реализуем эту структуру**
- * С одной стороны, такая структура нужна в хозяйстве для самых разных задач
- * А с другой, на её примере мы **увидим работу сбалансированного дерева**

ДОБАВЛЕНИЕ

- * Новый элемент добавляется в конец (в массиве — это конец массива)
- * Элемент сравнивается со своим родителем, и если он больше (для max-heap) или меньше (для min-heap), то **они меняются местами**
- * Операция **повторяется рекурсивно** (просеивание вверх / sift up)

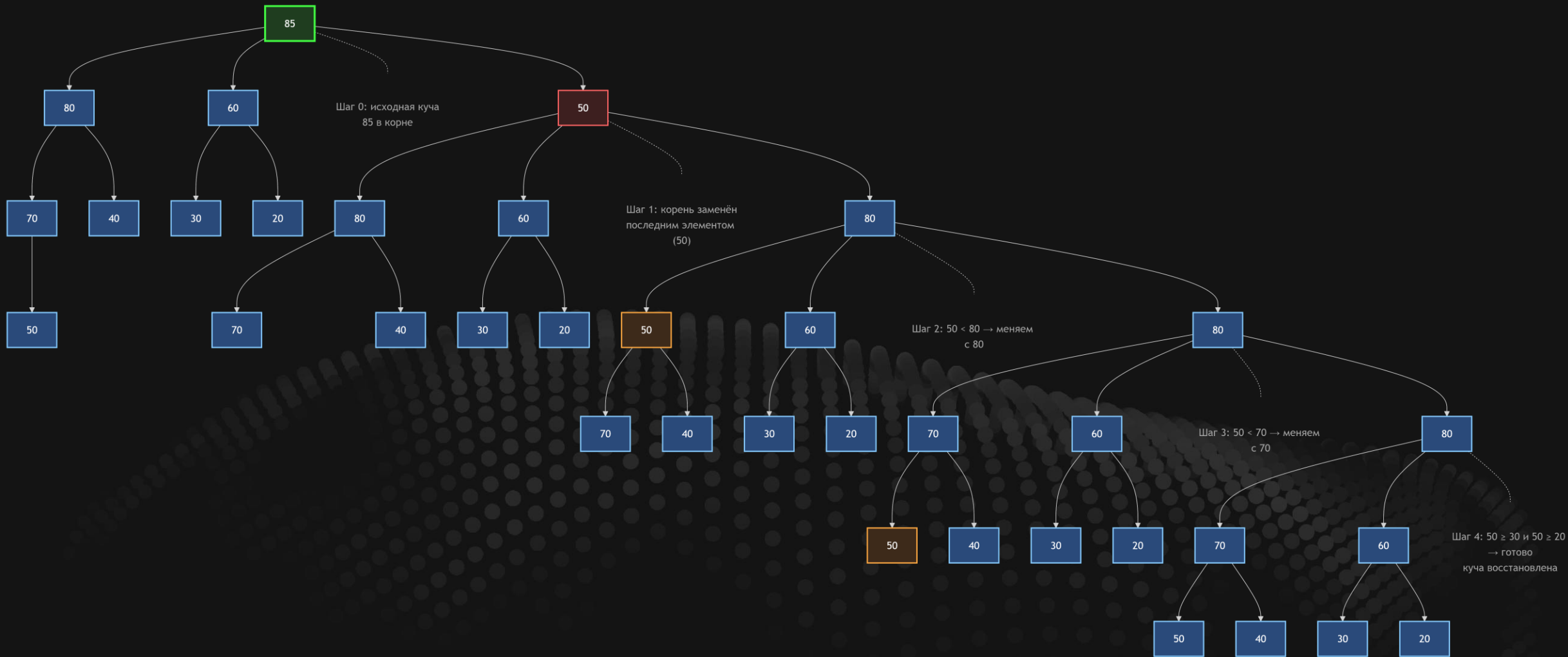


РЕФЛЕКСИРУЕМ

- * Легко увидеть, что **каждая операция всплытия «делит» нашу работу пополам** — поэтому вставка имеет математически доказанную логарифмическую сложность
- * При этом добавление в конец гарантирует, что **дерево всегда остаётся сбалансированным** — все его ветви имеют примерно одинаковую высоту
- * Более того, бинарная куча является «полным» деревом, **поэтому высота может отличаться только на единицу**

УДАЛЕНИЕ

- * Алгоритм похож на добавление, но **работает в обратную сторону**
- * Извлекаем корень и **заменяем его последним элементом дерева** (последним элементом массива)
- * **Просеивание вниз**: родитель меняется с тем потомком, который нарушает свойство кучи (в max-heap – с большим, в min-heap – с меньшим)
- * **Повторяем рекурсивно** (просеивание вниз / sift down)



СНОВА РЕФЛЕКСИРУЕМ

- * Удаление похоже на вставку, но наоборот
- * Обе операции выполняются за логарифмическое время $O(\log n)$
- * Используя эти свойства, можно реализовать полноценную сортировку массива с гарантированной сложностью $O(n \log n)$ — это пирамидальная сортировка (Heapsort)
- * Но в реальности из-за больших прыжков по массиву кэш используется не так эффективно, а также алгоритм сложно распараллелить
- * Поэтому быстрая сортировка (Quicksort) на практике всё равно оказывается быстрее (хоть и может деградировать)

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

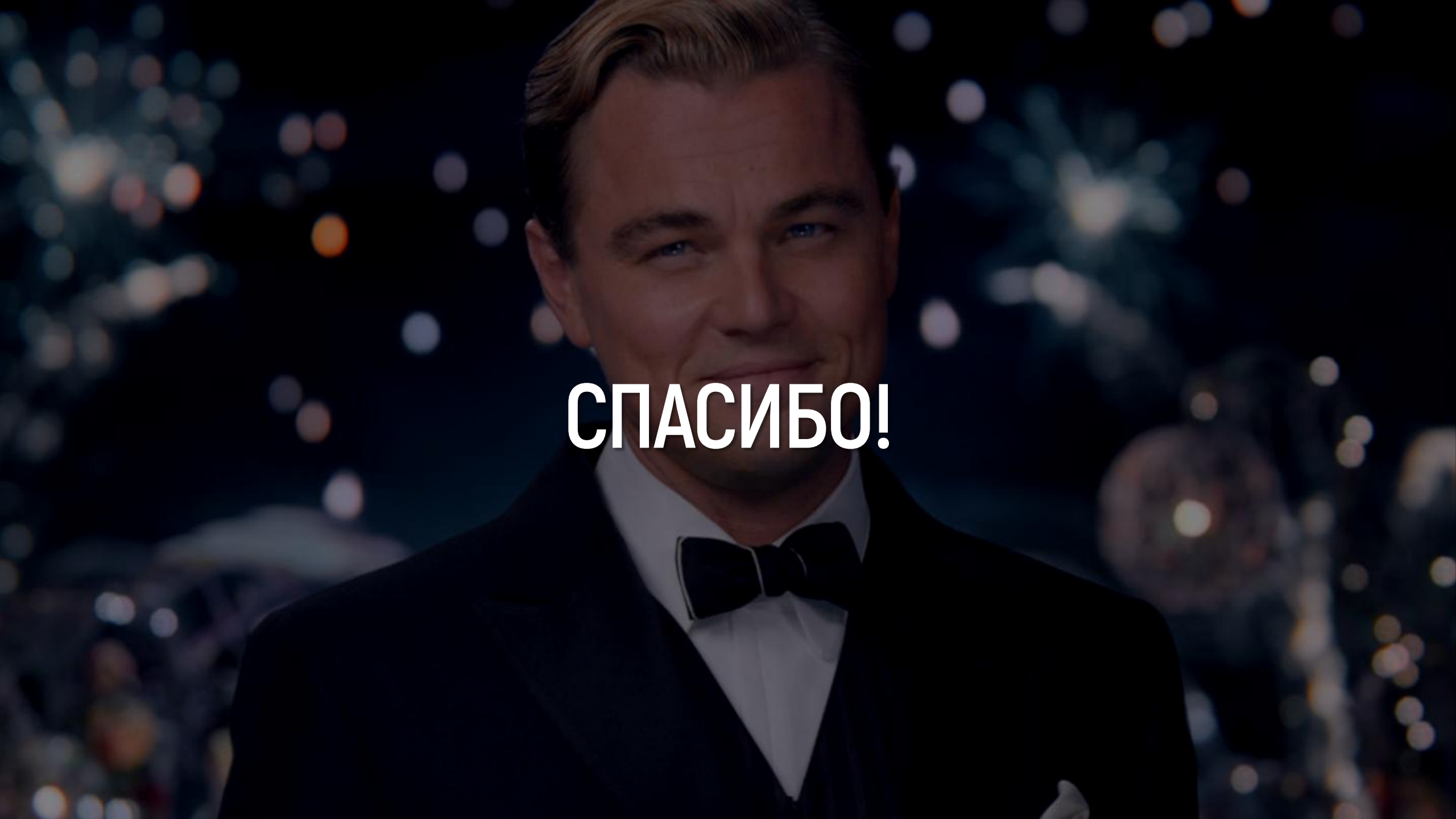
СТОЯНА, СТОЯНААА!

БЕЗ ПАНИКИ

- ✱ Сейчас мы откроем редактор и реализуем эту структуру самостоятельно
- ✱ Её реализация не потребует большого количества кода, но понимание даст куда больше

ПОДВЕДЁМ ИТОГИ

- * Сбалансированные деревья могут обслуживать **не только задачи поиска по ключу**
- * Бинарная куча — это хороший пример такого дерева, которое позволяет иметь **отсортированный набор данных с константным чтением корня $O(1)$**
- * Структура, которая реализуется с помощью бинарной кучи, называется **очередью с приоритетом**
- * Это **не единственная её реализация** (существуют биномиальные кучи, кучи Фибоначчи и другие)

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with out-of-focus bokeh lights in various colors (white, yellow, orange, blue), suggesting a night-time event or celebration with fireworks or city lights.

СПАСИБО!